

Sun, Aug 23 at 9:28 AM

Pasted markdown(1).md

File

I am trying to reproduce an old bioinformatics workflow and am missing Gene Ontology files. The files most likely came from 2015 or 2016 but I am not confident. One type of files maps Entrez Gene IDs to UniProt, or vice versa. The other provides annotations for genes. The archives at <https://ftp.ebi.ac.uk/pub/databases/GO/goa/old/HUMAN/> give one potential lead. I copied in the listing. Files in <https://release.geneontology.org/2016-06-01/annotations/gp2protein/index.html> or similar directories may also be relevant. I am not sure. Explore further. Another potential source of data is <https://ftp.ncbi.nlm.nih.gov/gene/DATA/>. Furthermore, you could explore historical releases of UniProt, Entrez Gene, or other reliable gene and protein databases to see what archives they have. Perform a deep search to find as many versions of these files as possible. Then we will start to assemble them. You may need to look at third party databases, GitHub repositories, web archives, zip files from research manuscripts, etc. The files will be hard to find. I have already tried to get them.

Findings

The evidence points to **three distinct historical products** that can easily be conflated:

1. **GO annotation records** — usually GAF or GPAD.
2. **Gene-product metadata** — GPI, formerly named gp_information.
3. **Identifier mappings** — Entrez Gene ↔ UniProt, generally supplied by UniProt, NCBI Gene, HGNC, Ensembl, or a packaged derivative such as Bioconductor.

The most likely original files depend on whether the workflow was keyed by UniProt accessions or Entrez Gene IDs.

Most likely original combinations

1. A UniProt-keyed GOA workflow

The leading candidate pair is:

- `gene_association.goa_human.<release>.gz` for GO annotations.
- A UniProt idmapping file, `NCBI gene2accession.gz`, or HGNC-derived table for Entrez↔UniProt conversion.

The GOA README says the annotation file's object ID is a Swiss-Prot/TrEMBL accession or Ensembl peptide ID, and the file contains GO term, evidence, aspect, taxon, and other annotation fields.

The uploaded EBI listing has a complete run from:

- gene_association.goa_human.140.gz — January 5, 2015
- through gene_association.goa_human.157.gz — May 9, 2016.

Starting at release 158 in June 2016, the same product was renamed:

- goa_human.gaf.158.gz
- continuing through at least goa_human.gaf.163.gz in November 2016.

This is my **highest-confidence interpretation** if the old workflow ingested a “gene association” file and then converted UniProt IDs to Entrez IDs.

2. A GPAD/GPI workflow

Another strong possibility is a matched pair:

- gp_association.goa_human.<release>.gz — GO assertions in GPAD form.
- gp_information.goa_human.<release>.gz — corresponding gene-product information.

Both series run from release 140 through 157 in the target period.

At release 158, these became:

- goa_human.gpa.158.gz
- goa_human.gpi.158.gz

The GPI sequence begins in June 2016 and continues as .159, .160, .161, .162, and .163.

This is especially likely when the old code expected **two complementary files**—one containing associations and another describing the annotated objects. GPI should not automatically be treated as a complete Entrez↔UniProt crosswalk, however; the historical GPI xref field needs to be inspected before relying on it for that purpose.

3. An Entrez-keyed NCBI workflow

The likely pair is:

- gene2go.gz for Entrez Gene ↔ GO annotations.

- gene2accession.gz for GeneID ↔ sequence and protein accessions.

NCBI documents gene2go as a GeneID/GO-term table containing evidence and GO category. gene2accession maps GeneIDs to genomic, RNA, and protein accessions and can include Swiss-Prot records. A second reconstruction path is gene2refseq.gz joined to gene_refseq_uniprotkb_collab.gz.

The difficulty is archival: NCBI's Gene DATA area is a **rolling, frequently recalculated publication area**, not a normal release archive. I did not locate byte-for-byte official 2015 or 2016 snapshots of gene2go.gz or gene2accession.gz.

Strongest exact Entrez↔UniProt snapshot

UniProt still publishes two explicit March 2015 snapshots:

- idmapping_selected.tab.2015_03.gz
- idmapping.dat.2015_03.gz

The selected table has:

- UniProt accession in column 1
- Entrez Gene/GeneID in column 3
- GO IDs in column 7
- NCBI taxon in column 13

Filtering column 13 for taxon 9606 produces the human subset. UniProt explicitly identifies the .2015_03 files as archived March 2015 data.

This is the best surviving **official standalone mapping file** I found.

UniProt also retains monthly previous-release directories throughout 2015 and 2016. Those knowledgebase releases can be used to reconstruct GeneID↔UniProt mappings from DR GeneID; records if a separate historical organism-specific mapping table is unavailable.

I did not locate an official historical copy of a file such as HUMAN_9606_idmapping.dat.gz or HUMAN_9606_idmapping_selected.tab.gz from the target dates.

Best practical all-in-one recovery source

The most useful recoverable substitute is probably **Bioconductor's historical org.Hs.eg.db packages**. They contain both:

- org.Hs.egUNIPROT — Entrez Gene ↔ UniProt accessions.

- org.Hs.egGO — Entrez Gene ↔ GO terms, including ontology and evidence information.

Four exact package snapshots span the period:

Package	Entrez source date	GO source date
org.Hs.eg.db 3.1.2	March 17, 2015	March 14, 2015
org.Hs.eg.db 3.2.3	September 27, 2015	September 19, 2015
org.Hs.eg.db 3.3.0	March 14, 2016	March 5, 2016
org.Hs.eg.db 3.4.0	September 26, 2016	September 21, 2016

The package archive pages confirm the exact versions and downloadable source tarballs.

The manuals explicitly document the Entrez→UniProt map and the dated GO sources. The March 2015, September 2015, March 2016, and September 2016 source dates are recorded in those manuals.

These are the **best reproducibility anchors** because each package preserves both missing data classes inside one immutable artifact.

What human.xrefs is—and why it is probably not the 2015/16 file

The historical GOA README describes xrefs.goa/human.xrefs as a tab-delimited bridge among Swiss-Prot/TrEMBL/Ensembl, IPI, RefSeq, HGNC, nucleotide protein identifiers, and legacy NCBI LocusLink.

However, the uploaded archive listing ends with:

- human.xrefs.98.gz — May 31, 2011
- human.xrefs.99.gz — June 10, 2011.

Therefore:

- It is an excellent clue about the **kind of mapping** the workflow might have expected.
- It is not a native 2015/2016 release series.
- It remains possible that the old workflow reused a stale human.xrefs.99.gz, particularly if it originated from an older pipeline.

What the GO gp2protein directories contain

GO's documentation says annotations/gp2protein contains mappings from gene-product identifiers—"usually MOD IDs"—to UniProtKB accessions. The archive runs back to 2004 and was reconstructed from the former GO CVS, SVN, and old product archives.

The corresponding GO SVN history identifies files for:

- FlyBase
- MGI
- RGD
- ZFIN
- EcoCyc
- DictyBase
- GeneDB organisms

I found no evidence of a human-wide gp2protein.human file. Consequently, this branch is **secondary for a human workflow**. It becomes relevant only when the original pipeline used a model-organism database identifier or processed multiple species.

Additional exact-release reconstruction sources

Ensembl releases 79–87

Ensembl retains releases 79 through 87, covering roughly March 2015 through December 2016. Releases 79, 83, and 87 are dated March 2015, December 2015, and December 2016 respectively, and each contains MySQL dumps.

The human core dumps contain the tables needed to reconstruct both identifier and ontology mappings:

- external_db
- xref
- object_xref
- ontology_xref
- gene
- transcript
- translation

The exact human release directories remain available.

This is more work than using Bioconductor, but it is a robust byte-preserved fallback.

HGNC quarterly snapshots

HGNC states that it archives complete-set files monthly and quarterly; old monthly files are deleted after 365 days, but quarterly files are retained. The complete set includes both `entrez_id` and `uniprot_ids`.

This is a strong potential Entrez↔UniProt source. I was not able to resolve the exact 2015/2016 Google Cloud object names from HGNC's JavaScript-dependent archive browser, so I have marked it as a confirmed archive with unresolved filenames rather than claiming files I could not verify.

Recommended reconstruction anchors

A sensible initial set is:

Approximate target	GOA annotation	Combined mapping fallback
March 2015	releases 142 or 143	org.Hs.eg.db 3.1.2 plus UniProt 2015_03
September 2015	release 148	org.Hs.eg.db 3.2.3
March 2016	release 155	org.Hs.eg.db 3.3.0
June 2016	release 158, first new GAF/GPA/GPI naming	nearest March or September Bioconductor package
September 2016	release 160	org.Hs.eg.db 3.4.0
November 2016	release 163	org.Hs.eg.db 3.4.0

Comparing outputs from those four dates should make it possible to identify the original vintage from gene counts, mapping counts, UniProt accession churn, or GO annotation totals.

Complete inventory

I parsed the uploaded listing into a reusable report containing:

- **180 exact 2015–2016 EBI GOA files**, including full, reference-proteome, complex, isoform, and RNA series.
- **61 surviving human.xrefs versions**.
- All 24 GO monthly release roots for 2015–2016.
- UniProt March 2015 mapping snapshots and monthly previous-release directories.
- NCBI candidate files and required joins.
- The four exact Bioconductor packages and their underlying source dates.
- Ensembl release roots and reconstruction tables.
- Remaining archive gaps and dead ends.

Download the historical GO and identifier-mapping inventory

historical_go_mapping_inventory.mdDocumentOpen file

 Citations



[EBI FTP](#)



[UniProt FTP](#)

https://ftp.uniprot.org/pub/databases/uniprot/previous_releases/release-2016_06/knowledgebase/





[Gene Ontology Resource](#)

<https://geneontology.org/docs/go-archives/>

?

[UniProt FTP](#)

[Index of /pub/databases/uniprot/previous_releases](#)

?



[Bioconductor Mirror](#)

<https://bioconductor.statistik.tu-dortmund.de/packages/3.1/data/annotation/html/org.Hs.eg.db.html>

?



[Bioconductor Mirror](#)

<https://bioconductor.statistik.tu-dortmund.de/packages/3.1/data/annotation/manuals/org.Hs.eg.db/man/org.Hs.eg.db.pdf>

?

[Ensembl FTP](#)

<https://ftp.ensembl.org/pub/>

?

[Ensembl FTP](#)

https://ftp.ensembl.org/pub/release-79/mysql/homo_sapiens_core_79_38/

?



[GeneNames](#)

[HGNC data archive help | HUGO Gene Nomenclature Committee](#)

Sun, Aug 23 at 10:11 PM

investigation_summary_2026_08_23.md

File

Greene2015_Table9.xlsx

Spreadsheet

Greene2015_Table6.xlsx

Spreadsheet

Greene2015_sup.pdf

PDF

Greene2015.pdf

PDF

OhmNet.pdf

PDF

msigdb_v6.0_files_to_download_locally.zip

Zip Archive

msigdb_v5.1_files_to_download_locally.zip

Zip Archive

msigdb_v5.2_chip_files_to_download_locally.zip

Zip Archive

msigdb_v5.2_files_to_download_locally.zip

Zip Archive

graphsage_ppi.zip

Zip Archive

dgl_ppi.zip

Zip Archive

bio-tissue-readme.txt

Document

bio-tissue-labels.tar.gz

File

bio-tissue-hierarchy.tar.gz

File

bio-tissue-networks.tar.gz

File

The attached investigation summary describes the pipeline we need to reproduce. It involves where the PPI dataset came from in the GraphSAGE manuscript and how it was transformed by the DGL package downstream. The motivation section explains. Several mapping questions have been resolved. Some remain open, especially related to the gene to GO term mapping. Your task is to first independently verify the questions that are claimed to be resolved. Do not make assumptions, rederive and verify all of the data transformations. Then continue to explore how the gene IDs were mapped to GO labels. Use as many combinations of the gene and GO mapping files you found above as needed. Do not trust the investigation summary's claims about why this mapping has not been resolved yet. I attached some relevant files. Make sure you can either run all analyses from these files or obtain reliable sources. Track all the sources of your input files so you can

create a report or download script later for the relevant files. Ask me to upload any files you cannot obtain. bio-tissue-* files are from OhmNet.

You're right—the attachment you received was evidently stale even though my working copy had the change. I've created a **new file under a new filename** so there's no ambiguity.

This version has explicit datestamp support and was tested end-to-end. For example:

```
python make_local_inventory_v2.py /path/to/files \
```

```
--output local_upload_inventory.csv \
```

```
--datestamp 20260827T095700Z
```

produces:

```
local_upload_inventory_20260827T095700Z.csv
```

Without --datestamp, it uses the current UTC time automatically.

Download the corrected script: make_local_inventory_v2.py

SHA-256:

```
a5fd20e62d0a9ad765436683b1d6bb294f07fc62a10f74b2187d40600c75cc88
```

I also verified that the script compiles and that the datestamped output file is actually created.

```
make_local_inventory_v2.pyCodeOpen file
```

```
local upload inventory 20260827T145903Z
```

artifact_name	relative_path	size_bytes	sha256
2016-06-01-go.obo	2016-06-01-go.obo	34762536	9b4c0c28d73ba41ae4c684d78b354d2c8bea691a5d759d4cdd188eecdd307ca2
2016-06-01-gp2protein.neid.gz	2016-06-01-gp2protein.neid.gz	38540366	f3a2d329ada32f03e4c3ec60c55ef77cfe3626c3c875832e59612e5f316504e7
2016-12-23-gene2go.gz	2016-12-23-gene2go.gz	19731322	33bedddfa375a99557fc8ddc8192b8441745f7c338714441ec3bafbed6228386

goa_human.gaf.159.gz

File

goa_human.gpa.159.gz

File

goa_human.gpi.159.gz

File

These are all the files I have locally, independent of you pattern filters \$ ls -l total 2618029 -rw-r--r-- 1 agitter 1049089 1999 Aug 15 14:38 2016-06-01-annotations-README -rw-r--r-- 1 agitter 1049089 30449937 Aug 15 14:16 2016-06-01-gene_ontology.obo -rw-r--r-- 1 agitter 1049089 34762536 Aug 15 14:16 2016-06-01-go.obo -rw-r--r-- 1 agitter 1049089 170536280 Aug 15 14:16 2016-06-01-go.owl -rw-r--r-- 1 agitter 1049089 38540366 Aug 15 14:45 2016-06-01-gp2protein.geneid.gz -rw-r--r-- 1 agitter 1049089 378274 Aug 15 14:20 2016-06-01-gp2protein.human.gz -rw-r--r-- 1 agitter 1049089 19731322 Aug 15 21:58 2016-12-23-gene2go.gz -rw-r--r-- 1 agitter 1049089 3280383 Aug 15 21:58 2016-12-23-gene2go_human.tsv.gz -rw-r--r-- 1 agitter 1049089 1374437901 Aug 15 21:42 2026-08-14-gene2go.gz -rw-r--r-- 1 agitter 1049089 5328626 Aug 15 21:50 2026-08-14-gene2go_human.tsv.gz -rw-r--r-- 1 agitter 1049089 1729378 Aug 16 08:32 Greene2015.pdf -rw-r--r-- 1 agitter 1049089 37992 Aug 16 08:34 Greene2015_Table6.xlsx -rw-r--r-- 1 agitter 1049089 140272 Aug 16 08:34 Greene2015_Table9.xlsx -rw-r--r-- 1 agitter 1049089 6701167 Aug 16 08:33 Greene2015_sup.pdf -rw-r--r-- 1 agitter 1049089 8897160 Aug 16 16:44 HuamnBase-kidney.dat -rw-r--r-- 1 agitter 1049089 8401428 Aug 16 16:43 HumanBase-blood.dat -rw-r--r-- 1 agitter 1049089 182612266 Aug 16 16:47 HumanBase-blood_top.gz -rw-r--r-- 1 agitter 1049089 1458562 Aug 16 08:31 OhmNet.pdf -rw-r--r-- 1 agitter 1049089 421785 Aug 14 11:14 bio-tissue-hierarchy.tar.gz -rw-r--r-- 1 agitter 1049089 1793328 Aug 14 11:14 bio-tissue-labels.tar.gz -rw-r--r-- 1 agitter 1049089 15568869 Aug 14 11:12 bio-tissue-networks.tar.gz -rw-r--r-- 1 agitter 1049089 1465 Aug 14 11:18 bio-tissue-readme.txt -rw-r--r-- 1 agitter 1049089 61468422 Aug 17 10:45 bioconductor-annotation-org.Hs.eg.db_3.0.0.tar.gz -rw-r--r-- 1 agitter 1049089 65439359 Aug 17 10:45 bioconductor-annotation-org.Hs.eg.db_3.1.2.tar.gz -rw-r--r-- 1 agitter 1049089 69122902 Aug 16 07:54 bioconductor-annotation-org.Hs.eg.db_3.3.0.tar.gz -rw-r--r-- 1 agitter 1049089 69557350 Aug 16 07:53 bioconductor-annotation-org.Hs.eg.db_3.4.0.tar.gz -rw-r--r-- 1 agitter 1049089 6597 Aug 17 16:14 blood_sample.tsv.gz -rw-r--r-- 1 agitter 1049089 55969 Aug 23 11:02 chat_gpt_historical_go_mapping_inventory.md -rw-r--r-- 1 agitter 1049089 7088822 Aug 14 21:50 dgl_ppi.zip -rw-r--r-- 1 agitter 1049089 147142 Aug 21 15:37 dhimmel-gene-ontology-962a5e1-GO_annotations-9606-direct-allev.tsv -rw-r--r-- 1 agitter 1049089

150794 Aug 21 15:37 dhimmel-gene-ontology-962a5e1-GO_annotations-9606-direct-expev.tsv -rw-r--r-- 1 agitter 1049089 147185 Aug 21 15:37 dhimmel-gene-ontology-962a5e1-GO_annotations-9606-inferred-allev.tsv -rw-r--r-- 1 agitter 1049089 147186 Aug 21 15:37 dhimmel-gene-ontology-962a5e1-GO_annotations-9606-inferred-expev.tsv -rw-r--r-- 1 agitter 1049089 194113 Aug 18 13:16 file_inventory_2026_08_18.pdf -rwxr-xr-x 1 agitter 1049089 578 Aug 15 21:48 filter_gene2go.sh* -rwxr-xr-x 1 agitter 1049089 7907 Aug 22 19:22 find_gene2go_2016.py* -rw-r--r-- 1 agitter 1049089 3160 Aug 22 19:27 gene2go_github_survey.tsv -rw-r--r-- 1 agitter 1049089 6546187 Aug 15 15:06 gene_association.goa_human.155.gz -rw-r--r-- 1 agitter 1049089 6466998 Aug 15 21:24 gene_association.goa_human.156.gz -rw-r--r-- 1 agitter 1049089 6434409 Aug 15 21:24 gene_association.goa_human.157.gz -rw-r--r-- 1 agitter 1049089 4859536 Aug 27 09:32 gene_association.goa_ref_human.157.gz -rw-r--r-- 1 agitter 1049089 4854158 Aug 15 21:25 goa_human.gaf.158.gz -rw-r--r-- 1 agitter 1049089 4890788 Aug 15 11:24 goa_human.gaf.159.gz -rw-r--r-- 1 agitter 1049089 4574158 Aug 15 11:24 goa_human.gaf.160.gz -rw-r--r-- 1 agitter 1049089 3636575 Aug 27 09:31 goa_human.gpa.158.gz -rw-r--r-- 1 agitter 1049089 3665510 Aug 27 09:30 goa_human.gpa.159.gz -rw-r--r-- 1 agitter 1049089 3370602 Aug 27 09:31 goa_human.gpa.160.gz -rw-r--r-- 1 agitter 1049089 602496 Aug 27 09:31 goa_human.gpi.158.gz -rw-r--r-- 1 agitter 1049089 602689 Aug 15 14:04 goa_human.gpi.159.gz -rw-r--r-- 1 agitter 1049089 693073 Aug 15 14:04 goa_human.gpi.160.gz -rw-r--r-- 1 agitter 1049089 4183687 Aug 27 09:32 gp_association.goa_human.157.gz -rw-r--r-- 1 agitter 1049089 27029260 Aug 14 21:52 graphsage_ppi.zip -rw-r--r-- 1 agitter 1049089 13819 Aug 23 21:59 investigation_summary_2026_08_23.md -rw-r--r-- 1 agitter 1049089 3452 Aug 27 09:59 local_upload_inventory_20260827T145903Z.csv -rwxr-xr-x 1 agitter 1049089 7070 Aug 27 09:54 make_local_inventory.py* -rwxr-xr-x 1 agitter 1049089 9202 Aug 27 09:58 make_local_inventory_v2.py* -rw-r--r-- 1 agitter 1049089 115484475 Aug 15 15:04 msigdb_v5.0_files_to_download_locally.zip -rw-r--r-- 1 agitter 1049089 69702250 Aug 15 09:17 msigdb_v5.1_files_to_download_locally.zip -rw-r--r-- 1 agitter 1049089 67891640 Aug 15 08:53 msigdb_v5.2_chip_files_to_download_locally.zip -rw-r--r-- 1 agitter 1049089 92768160 Aug 15 08:53 msigdb_v5.2_files_to_download_locally.zip -rw-r--r-- 1 agitter 1049089 72144318 Aug 15 09:17 msigdb_v6.0_files_to_download_locally.zip -rw-r--r-- 1 agitter 1049089 1092580 Aug 15 09:35 ohmnet-master.zip -rw-r--r-- 1 agitter 1049089 1164 Aug 27 09:55 requested_file_patterns.txt -rwxr-xr-x 1 agitter 1049089 2110 Aug 17 16:09 sample_giant_network.sh* -rw-r--r-- 1 agitter 1049089 455680 Aug 14 23:05 train_graph_id.npy If you do not see a file there, I need to download it locally. I will keep the csv because it doesn't free substantial space. Here are the next files.

Thu, Aug 27 at 11:04 AM

2016-06-01-annotations-README

File

2016-06-01-gp2protein.geneid.gz

File

2016-06-01-gp2protein.human.gz

File

local upload inventory full 20260827T160408Z

artifact_name	relative_path	size_bytes	sha256
2016-06-01-annotations-README	2016-06-01-annotations-README	1999	32134f9555d6710a9bb488fe652fef04cd84facf516f669939487d137f8bcc88
2016-06-01-gene_ontology.obo	2016-06-01-gene_ontology.obo	30449937	3bc7e81ebac6f6e99b75b136ae0c97542b711697784ffb0275575afdd5a0298c
2016-06-01-go.obo	2016-06-01-go.obo	34762536	9b4c0c28d73ba41ae4c684d78b354d2c8bea691a5d759d4cdd188eecdd307ca2

Deleted B101

Thu, Aug 27 at 3:21 PM

idmapping_2026_08_27.tsv.gz

File

2016-06-01-go.obo

File

Deleted B102 UniProt currently shows P0DO97 · CC192_HUMAN with GeneID 728586. It may have changed since 2016. You need to account for ambiguous mappings. That is common in bioinformatics with many-to-many mappings possible. I used the current UniProt ID mapping tool on 176, 337, 3108, 4018, 7957, 10159, 29901, 55125, 84919. See

the uploaded file idmapping_2026_08_27.tsv.gz They may not be consistent with 2016 mappings. B103 is here.

Thu, Aug 27 at 9:52 PM

goa_human.gpi.158.gz

File

goa_human.gaf.158.gz

File

goa_human.gpa.158.gz

File

This appears to be a major breakthrough! Excellent work. Let's continue to be careful and skeptical.

We should track the GeneID 7957 and 29901 mappings that have changed since GPI v159 to double check correctness. We so not want to make incorrect assumptions.

Do I need to further explore GeneID 10159, ATP6AP2?

<https://www.ncbi.nlm.nih.gov/gene/?term=10159> shows

mRNA and Protein(s)

1. [NM_005765.3](#)** → [NP_005756.2](#) renin receptor precursor**

[See identical proteins and their annotated locations for NP_005756.2](#)

Status: REVIEWED

Source sequence(s)

[AC092473](#)

Consensus CDS

[CCDS14252.1](#)

UniProtKB/Swiss-Prot

[B7Z9I3](#), [O75787](#), [Q5QIQ7](#), [Q6T7F5](#), [Q8NBP3](#), [Q8NG15](#), [Q96FV6](#), [Q96LB5](#), [Q9H2P8](#), [Q9UG89](#)

UniProtKB/TrEMBL

[A0A1C7CYW4](#)

Related

[ENSP00000490083.1](#), [ENST00000636580.2](#)

Deleted B103

We need to continue pushing on these remaining GO labels. Explore this next data batch as well as other possibilities.

Friday 12:52 PM

`msigdb_v5.0_files_to_download_locally.zip`

Zip Archive

Excellent progress. Here is the v5.0 MSigDB file. I had to delete the older MSigDB files to make space. I can upload them again later if needed. Continue exploring the column order and other features that could provide this order. That would disambiguate the columns with identical values. We will return to the UniProt FTP downloads because of storage space limitations. If you want me to test them, provide a single script to download each, run that extraction, and delete the full large file. We need it to record the data provenance of course. We will have to defer B105 again.

Friday 5:29 PM

uniprot_2016_mapping.zip

Zip Archive

I am getting reports from you that my storage is full even though my settings show 72.3 MB of 512 MB used. This is my library sorted by file size. Deleted B104C The column ordering is another splendid result. Is that the best we can do, or other there other variations that would perfectly match all columns? Consider the entire likely range of dates when this analysis may have been performed. Good job designating the column disambiguation as likely, not proven. We need to carefully support all of our claims and moderate their strength appropriately. uniprot_2016_mapping.zip has the script outputs.

Saturday 7:07 AM

uniprot_2016_mapping_audit_ledger.zip

Zip Archive

goa_date_screen_results.zip

Zip Archive

uniprot_2016_mapping_audit_ledger.zip now has the requested subdirectories inside. I also prepared the GOA date screen results.

Saturday 8:51 AM

dhimmel-gene-ontology-962a5e1-GO_annotations-9606-direct-allev.tsv

File

dhimmel-gene-ontology-962a5e1-GO_annotations-9606-direct-expev.tsv

File

dhimmel-gene-ontology-962a5e1-GO_annotations-9606-inferred-allev.tsv

File

dhimmel-gene-ontology-962a5e1-GO_annotations-9606-inferred-expev.tsv

File

Are you suggesting that you know the procedure for selecting the MSigDB features, or is that speculative?

The chr9q11 entry is important and strong support for this hypothesis. It would otherwise be strange to have empty columns. This suggests there was not much data inspection after

the features were created. We should still note this as a strong hypothesis, not definitive evidence.

Here are the GO annotation files requested.

Deleted B104E

I am attempting to have another agent independently verify this analysis. It got stuck and asked:

1. Which gene2go or GOA version did you use to achieve 121/121 exact? You mention GOA v159 but my independent reproduction gets 24/121 exact with GOA v159.
2. For the 12 genes I identify as error sources (3248, 3988, 8564, 27201, 30061, 51166, 51312, 55471, 55801, 56994, 79017, 121599), how did you resolve them? Did you use a different ID mapping source, or a different annotation file?
3. Did you verify that your 121/121 result uses a single consistent mapping, or did you tune per-gene assignments to maximize the match?
4. The GPI file — you mention using it for ID mapping. Can you specify exactly which GPI fields you used and how they differ from gp2protein.geneid.gz?

Saturday 9:57 AM

GO_annotations-9606-direct-expev.tsv

File

GO_annotations-9606-inferred-expev.tsv

File

GO_annotations-9606-inferred-allev.tsv

File

GO_annotations-9606-direct-allev.tsv

File

If the 121/121 match does not cover all GraphSAGE rows, we need to be more careful in how we declare this to be a success. I misunderstood and though we had a perfect mapping. Review again what remains ambiguous across all parts of the GraphSAGE input. Is there any opportunity to continue disambiguating that? Deleted B104F Here are the script outputs.

Canonical node-to-gene mapping

Yes. The recovered ordering gives a simple, directly usable mapping from every anonymous GraphSAGE node to an Entrez Gene ID.

The compact table contains:

graphsage_node_id

feature_label_row_index

graph_index_1based

tissue

split

local_node_index_0based

entrez_gene_id

Because ppi-id_map.json is the identity mapping, graphsage_node_id is also the row index in ppi-feats.npy and the key identifying the corresponding vector in ppi-class_map.json.

- Compact 56,944-row node-to-Entrez table
- Mapping schema and interpretation

I also generated an evidence-rich table. For each GraphSAGE row it records:

- the source OhmNet tissue and edgelist;
- the decompressed SHA-256 of that tissue edgelist;
- the CPython 2 dictionary slot and table size;
- the basis of the earlier independent mapping;
- GraphSAGE and OhmNet neighbor-set hashes;
- observed and independently reconstructed feature-vector hashes;
- observed and independently reconstructed GO-label-vector hashes;
- exactness flags for topology, features, and labels.
- Evidence-rich node-to-Entrez table
- Machine-readable validation summary
- Exact mapping-generation script

Evidence tiers

The mapping explicitly distinguishes three levels of row-identity evidence:

Tier Rows How identity was established

A	55,878	Independently resolved by graph topology alone
B	533	Independently resolved by topology plus reconstructed MSigDB features
C	533	Resolved by the inferred CPython 2 string-dictionary order, then cross-checked against topology, features, and GO labels

The important point is that the ordering mechanism was tested against **56,411 identities established without that mechanism** and disagreed with none of them. It then resolves the remaining 533 rows and gives:

- exact OhmNet neighbor sets for all 56,944 rows;
- exact reconstruction of all 2,847,200 feature cells;
- exact reconstruction of all 6,890,224 label cells.

The feature and label values were not used to optimize individual Tier C assignments. They are downstream cross-checks of one globally applied ordering rule.

How strong is this without the original scripts?

This is close to the strongest empirical conclusion available without a historical script, environment image, or author confirmation.

A careful final claim should separate two things:

Exact data reconstruction

This part is directly established from the files:

The released mapping deterministically reconstructs all 56,944 GraphSAGE PPI node identities and exactly reproduces every deposited edge, feature value, and label value.

Historical mechanism

This remains an inference:

The node order is best explained by reading the original OhmNet edgelist with node IDs left as strings, inserting them into an unrandomized 64-bit CPython 2 dictionary through legacy NetworkX graph construction, and iterating the dictionary table.

That mechanism is exceptionally well supported because:

- it matches all 56,411 independently anchored rows;
- obvious alternatives—numeric sorting, first appearance, integer-key dictionaries, reverse sorting—match only a few dozen rows;
- GraphSAGE explicitly required NetworkX no later than version 1.11, consistent with the reconstructed legacy environment.
- a separate simulation reproduces the complete 56,944-key order of the deposited class-map JSON.

Nevertheless, “the original authors’ code did exactly this” remains slightly stronger than the evidence permits. The appropriate phrase is **forensic reconstruction with exact out-of-sample confirmation**, not direct source-code proof.

For the final report, I recommend four evidence categories:

1. **Byte-level exact:** reproduced directly from available data.
2. **Strongly inferred:** one mechanism explains all data and survives independent controls.
3. **Documented:** stated by a manuscript or official source page.
4. **Open:** multiple histories remain observationally equivalent.

That framework will prevent small uncertainties such as ≥ 200 versus > 200 from being mixed with much stronger conclusions.

Should we verify the origin of the PPI edges?

Yes—but only one additional audit has high expected value.

What is already established

The complete GraphSAGE graph topology is exactly the topology of the 24 identified OhmNet tissue networks. This is stronger than merely relying on manuscript wording.

The OhmNet manuscript says its global unweighted physical interactome combined several resources, including MIntAct/IntAct, the Rolland interactome, BioGRID, HPRD, CORUM, and the interactome described by Menche et al. It reports 21,557 proteins and 342,353 experimentally supported physical interactions. Tissue-specific networks were then formed by filtering this global network using the Greene tissue-activity criterion.

Therefore, the old shorthand that the graphs are simply “BioGRID networks” is too narrow. BioGRID was one contributor to a composite physical interactome.

BioSNAP separately publishes:

- a combined OhmNet tissue network with 4,510 nodes and 70,338 unique edge pairs; and
- a global physical human PPI network with the same 21,557-node and 342,353-edge statistics reported in the OhmNet manuscript.

The worthwhile remaining edge audit

The final high-value test is:

1. Verify that the union of the supplied 144 OhmNet tissue edgelist equals the BioSNAP combined OhmNet file.
2. Verify that this union is a subset of the BioSNAP 342,353-edge global physical interactome.
3. Record source URLs, compressed and decompressed hashes, counts, and discrepancies.

The two downloads total only about **4.4 MB**. I prepared a single script that downloads them, runs both comparisons, creates a compact result package, and deletes the raw downloads only after every check succeeds:

- Download the BioSNAP PPI provenance-audit script

Run it with:

```
python download_and_verify_biosnap_ppi_sources.py \
--ohmnet-tar /path/to/bio-tissue-networks.tar.gz \
--graphsage-zip /path/to/graphsage_ppi.zip \
--output-dir biosnap_ppi_audit_results \
--delete-raw-after-package
```

This would establish the exact flattened global-network ancestry if it passes. It still would not tell us which individual database contributed each edge. Per-edge attribution to BioGRID versus IntAct, HPRD, CORUM, and the other sources would require an intermediate table containing source tags. That is much less likely to survive and is not necessary for the central GraphSAGE provenance result.

Harder search for copies containing label names

I searched more broadly across:

- the original GraphSAGE repository and issue history;
- GitHub forks and copied datasets;
- DGL, PyTorch Geometric, and DeepGNN;
- BioSNAP and OhmNet-related data;
- Zenodo, Figshare, Hugging Face, and Kaggle;
- exact GraphSAGE filenames;
- exact candidate GO identifiers;
- combinations of the PPI dimensions, GO terminology, and label-column names.

I did not find a public copy retaining the ordered names of all 121 labels.

This is consistent with the original repository history:

- In issue #78, a GraphSAGE collaborator stated that the graph came from OhmNet and that MSigDB supplied the feature and label families, but later requests for the preprocessing code were not answered.
- Issue #188 asks directly for a protein-ID mapping and has no published answer.
- Major downstream loaders still expose only 50 anonymous features and 121 anonymous labels.
- Teaching and challenge copies generally rename the targets label_0 through label_120, rather than preserving GO IDs.

BioSNAP does publish named OhmNet tissue–function–gene associations, but those are the OhmNet tissue-specific labels, not the 121 global GraphSAGE columns.

This does not prove that no named copy exists. It means further general web searching now has low expected return. The best remaining paths are:

- direct contact with the authors;
- old lab or cluster backups;
- an archived intermediate list of GO terms;
- unpublished preprocessing notebooks;

- a copy retained by an early GraphSAGE collaborator.

Those are primarily documentary routes, not computational ones.

Highest-value remaining provenance work

My current ranking is:

High value

1. **Run the small BioSNAP edge audit.**
This completes the external-source chain for graph topology at very low cost.
2. **Freeze the complete reconstruction as a minimal, rerunnable package.**
This should include exact hashes, raw-source acquisition, deterministic transformations, intermediate reconciliation counts, and assertions that fail loudly.
3. **Seek an ordered label-name artifact or author confirmation.**
This is the only direct way to prove the six duplicate-vector column names and the precise label-column ordering mechanism.

Medium value

4. **Resolve the 24-of-144 graph selection and 20/2/2 split rule.**
This matters for full historical reconstruction, but not for proving the severe leakage or identifying the source data.
5. **Compare an Entrez-native annotation source that yields the same effective labels.**
This could tell us whether GOA v159 was literally used or merely reproduces an equivalent intermediate table.

Low value now

6. **B105 ontology sensitivity.**
The current ontology already gives exact memberships. B105 is now a robustness check rather than a missing explanation.
7. **Distinguish MSigDB versions 5.0–6.0 from the feature matrix.**
They produce the same 50 vectors, so the matrix cannot settle the version.
8. **Resolve ≥ 200 versus >200 from the matrix alone.**
There is no set at the boundary that distinguishes those operators.

9. **Attribute every PPI edge to an individual contributing database.**

Interesting but unlikely to be recoverable from the flattened network and not needed for the main provenance finding.

The detailed assessment is here:

- Source-provenance assessment and remaining priorities

Leakage experiment saved

I saved the randomized-label experiment as a concrete analysis plan:

- Randomized-label leakage experiment backlog

The crucial design choice is to randomize at the **Entrez GeneID level**, then copy each randomized vector to every tissue occurrence of that gene. An independent row-wise shuffle would destroy the repeated-identity structure and would not test the leakage mechanism.

The planned demonstration has three central conditions:

1. Original biological labels with the GeneID lookup model.
2. Gene-level randomized labels with the same tissue split.
3. A gene-disjoint split in which no GeneID can appear in both training and evaluation.

A particularly strong control will permute complete label vectors separately among training-seen and training-unseen genes. That destroys biological correspondence while preserving the number of positive cells in each overlap stratum, so the lookup model's micro-F1 remains exactly unchanged. This will demonstrate algebraically—not just approximately—that the high score can survive removal of biological meaning.

Updated provenance package

- Final source ledger through B104H
- Append-only provenance events through B104H
- Actual-input manifest through B104H
- Final programmatic validation
- Complete node-mapping and provenance bundle

Bundle SHA-256:

975ad05d50558c8c0041e964805c7fc12aacf437178c354e45d45b75dfec3d94

The sensible transition is to run the small upstream PPI audit, then concentrate on the reproducible report and the randomized-label/gene-disjoint leakage demonstration.

GRAPHSAGE_NODE_MAPPING_README_20260830T110259Z.mdDocumentOpen file

B104H_MAPPING_VALIDATION_20260830T110259Z.jsonCodeOpen file

build_B104H_node_mapping_artifacts.pyCodeOpen file

download_and_verify_biosnap_ppi_sources.pyCodeOpen file

B104H_SOURCE_PROVENANCE_AND_NEXT_PRIORITIES_20260830T110259Z.mdDocume
ntOpen file

LEAKAGE_EXPERIMENT_BACKLOG_20260830T110259Z.mdDocumentOpen file

source_ledger_through_B104H_FINAL_20260830T110259Z.csvSpreadsheetOpen file

provenance_events_through_B104H_FINAL_20260830T110259Z.csvSpreadsheetOpen file

actual_input_file_manifest_through_B104H_20260830T110259Z.csvSpreadsheetOpen file

B104H_FINAL_DELIVERY_VALIDATION_20260830T110259Z.jsonCodeOpen file

7 more

 Citations





[GitHub](#)

[GitHub](#)

<https://github.com/williamleif/GraphSAGE>





[Computer Science](#)

<https://cs.stanford.edu/people/jure/pubs/ohmnet-ismb17.pdf>

?



[SNAP](#)

<https://snap.stanford.edu/biodata/datasets/10013/10013-PPT-Ohmnet.html>

?



[GitHub](#)

[GitHub](#)

<https://github.com/williamleif/GraphSAGE/issues/78>

?



[GitHub](#)

[GitHub](#)

<https://github.com/williamleif/GraphSAGE/issues/188>

?



[DGL](#)

https://www.dgl.ai/dgl_docs/en/2.3.x/modules/dgl/data/ppi.html

?



[GitHub](#)

[GitHub](#)

<https://github.com/naziish/GNN-PPI-challenge>

?



[SNAP](#)

<https://snap.stanford.edu/biodata/datasets/10014/10014-TFG-Ohmnet.tsv>

README_20260830T113352Z.md

File

biosnap_ppi_provenance_audit_20260830T113352Z.json

File

biosnap_ppi_provenance_audit_20260830T113352Z.zip

Zip Archive

The node mapping table has split. Does that contain one or multiple values? I agree with your four evidence categories. In addition, we need to document where our evidence differs from the literature, especially OhmNet and GraphSAGE. \$ python download_and_verify_biosnap_ppi_sources.py \ --ohmnet-tar bio-tissue-networks.tar.gz \ -graphsage-zip graphsage_ppi.zip \ --output-dir biosnap_ppi_audit_results \ --delete-raw-after-package { "results_zip": "biosnap_ppi_audit_results\\biosnap_ppi_provenance_audit_20260830T113352Z.zip", "results_zip_sha256": "adaf28f8bcbe2490d4af9ad212fc646d3ea183b315f0c95835cc3fa36f0df0c3", "checks_pass": true } Let's check the Menche paper for any information about the PPI network origin we can use. Make a note of GraphSAGE GitHub issues related to data provenance. Once our report is ready, we will respond to those to help old users.

Yes. Since the files are now safely in the repo and the destination subdirectories are under your control, **a few deliberate mv commands are the simplest approach**. I would avoid one giant set of broad globs, though, because filenames such as goa_date_screen_results/ are analysis outputs whereas goa_human.gaf.* are source data.

From:

```
cd /c/Users/agitter/Documents/students/neha_talluri/GraphSagePPIAnalysis/ai
```

I suggest this structure:

ai/

```
├── data/
|   ├── raw/
|   ├── reference/
|   └── derived/
├── gpt56sol/
└── other_agent/
```

Create it:

```
mkdir -p data/{raw,reference,derived} other_agent gpt56sol/user_runs
```

Then move the obvious **external/raw biological data** with a few globs:

```
mv 2016-06-01-* data/raw/
```

```
mv 2016-12-23-gene2go* 2026-08-14-gene2go* data/raw/
```

```
mv bio-tissue-* data/raw/
```

```
mv bioconductor-annotation-* data/raw/
```

```
mv gene_association.* goa_human.* gp_association.* data/raw/
```

```
mv msigdb_* data/raw/
```

```
mv graphsage_ppi.zip dgl_ppi.zip ohmnet-master.zip data/raw/
```

```
mv HumanBase-* HuamnBase-* blood_sample.tsv.gz data/raw/
```

```
mv idmapping_2026_08_27.tsv.gz data/raw/
```

```
mv GO_annotations-9606-*.tsv data/raw/
```

Move the manuscripts and supplements separately:

```
mv Greene2015.pdf Greene2015_sup.pdf Greene2015_Table6.xlsx  
Greene2015_Table9.xlsx data/reference/
```

```
mv OhmNet.pdf data/reference/
```

The UniProt extraction directories are **outputs of the scripts I prepared**, so I would keep them with the investigation rather than raw data:

```
mv 2016_04 2016_05 2016_06 gpt56sol/user_runs/
```

```
mv uniprot_2016_mapping_audit_ledger.* gpt56sol/user_runs/
```

Likewise for the two larger provenance runs:

```
mv biosnap_ppi_audit_results gpt56sol/user_runs/
```

```
mv goa_date_screen_results gpt56sol/user_runs/
```

```
mv goa_date_screen_results.zip gpt56sol/user_runs/
```

goa_date_screen_cache/ is disposable cache, so unless you specifically want it for forensic completeness:

```
rm -rf goa_date_screen_cache
```

For the older investigator's material, I would move these together:

```
mv investigation_summary_2026_08_23.md other_agent/
```

```
mv file_inventory_2026_08_18.pdf other_agent/
```

```
mv filter_gene2go.sh find_gene2go_2016.py gene2go_github_survey.tsv other_agent/
```

```
mv sample_giant_network.sh train_graph_id.npy other_agent/
```

I would **not yet use globs on the remaining root-level .py, .md, .csv, or .zip files**, because those mix my scripts/artifacts with other-agent artifacts and historical downloads. After the commands above, run:

```
find . -maxdepth 1 -type f -printf '%f\n' | sort
```

and paste that much shorter remainder here. I can give you the final exact mv commands without risking misclassification.

Also, the four files named:

```
dhimmel-gene-ontology-962a5e1-GO_annotations-9606-*.tsv
```

were the accidentally saved GitHub **HTML pages**, not genuine TSV data. Since you now have the genuine GO_annotations-9606-*.tsv files, I would delete the four bad copies rather than move them into the repo:

```
rm dhimmel-gene-ontology-962a5e1-GO_annotations-9606-*.tsv
```

That keeps data/raw/ limited to actual source data rather than failed downloads.

Yes. With the current root listing, almost everything remaining can now be classified confidently.

I would make these directories:

```
mkdir -p \
```

```
gpt56sol/scripts \
```

```
gpt56sol/provenance/dhimmel_download \
```

```
gpt56sol/provenance/local_inventories \
```

```
gpt56sol/reference_packs \
```

```
gpt56sol/user_runs \
```

```
gpt56sol/snapshot_packages
```

Then these are definitely mine or outputs of scripts I supplied:

```
mv B104D_GOA_date_screen_reference_pack_20260829T010311Z.zip \
```

```
gpt56sol/reference_packs/
```

```
mv download_and_verify_biosnap_ppi_sources.py \
```

```
download_extract_uniprot_2016_mapping_audit.py \
```

```
download_raw_dhimmel_annotations.py \
```

```
screen_goa_release_date_range.py \
```

```
make_local_inventory.py \
```

```
make_local_inventory_v2.py \
```

```
gpt56sol/scripts/
```



```
mv dhimmel_GO_annotations_962a5e1_manifest.csv \  
gpt56sol/provenance/dhimmel_download/
```

```
mv local_upload_inventory_20260827T145903Z.csv \  
local_upload_inventory_full_20260827T160408Z.csv \  
gpt56sol/provenance/local_inventories/
```

```
mv requested_file_patterns.txt \  
gpt56sol/user_runs/
```

Those classifications agree with the inventory I generated earlier: the download/audit/inventory programs are assistant-provided scripts; the local inventories, dhimmel manifest, reference pack, and requested-pattern file are outputs/artifacts from that work.

The four fake dhimmel TSVs

These four remaining files are the accidentally saved GitHub HTML pages, not scientific TSVs:

```
dhimmel-gene-ontology-962a5e1-GO_annotations-9606-direct-allev.tsv
```

```
dhimmel-gene-ontology-962a5e1-GO_annotations-9606-direct-expev.tsv
```

```
dhimmel-gene-ontology-962a5e1-GO_annotations-9606-inferred-allev.tsv
```

```
dhimmel-gene-ontology-962a5e1-GO_annotations-9606-inferred-expev.tsv
```

You already have the genuine raw files under their shorter GO_annotations-9606-... names. I would simply remove these bad copies:

```
rm dhimmel-gene-ontology-962a5e1-GO_annotations-9606-*.tsv
```

If you prefer retaining failed-download evidence, put them under gpt56sol/provenance/invalid_downloads/ instead. I don't think keeping the actual HTML bytes is necessary because their hashes and rejection are already documented in the GPT artifact history.

The 59 MB snapshot ZIP

These are also mine:

gpt56sol_complete_artifact_snapshot_20260830T124820Z.zip

gpt56sol_complete_artifact_snapshot_20260830T124820Z.sha256.txt

Since gpt56sol/ has already been extracted, the ZIP is now a redundant packaging artifact. I would either keep it outside Git entirely, or put it here:

```
mv gpt56sol_complete_artifact_snapshot_20260830T124820Z.* \  
gpt56sol/snapshot_packages/
```

I would **not commit the 59 MB ZIP to ordinary Git**. The extracted contents are much more useful for version control. The .sha256.txt can be committed even if the ZIP itself is gitignored or stored as a GitHub Release asset.

Your root README.md should remain at repository root.

After those moves, your top level should be essentially:

README.md

data/

gpt56sol/

opus46/

papers/

which is excellent.

What to ask Opus 4.6

The main thing we need from the other agent is **provenance, not scientific re-analysis**. We should be able to tell, file by file, whether something in opus46/ is:

- code written by Opus;
- an output produced by that code;
- an external/raw file it downloaded;
- a manually created note or report;
- a temporary/intermediate file;
- or a superseded result.

I would give it this exact request:

Please inventory every file currently under opus46/. For each file, report:

path, created_by, artifact_type, purpose, source/input files, source URL if externally downloaded, source version/date, command or script that created it, SHA-256, current vs superseded status, important findings supported, and recommended repository location.

In particular, identify any files under opus46/ that are actually unmodified external source data and therefore belong in data/ rather than opus46/. Also identify files that were supplied by GPT-5.6 Sol or generated by one of GPT-5.6 Sol's scripts and therefore belong under gpt56sol/.

Please do not infer provenance from filenames alone. Mark anything uncertain as uncertain.

There are several specific historical questions I would also have it answer:

1. Which analysis artifacts are uniquely Opus's?

Especially the early topology/WL mapping, initial GO-label matching, train_graph_id.npy, gene2go searches, HumanBase/GIANT experiments, and any DGL analysis.

2. Which external downloads did Opus initiate?

If a file is merely an untouched downloaded dataset, it belongs in data/, even if Opus was the agent that requested it.

3. Which results have been superseded?

Some early conclusions were later corrected substantially—for example, incomplete node identities, initial GO-label matching rates, MSigDB-version claims, and the description of the PPI source. Those files are valuable audit history but should be explicitly labeled historical/superseded.

4. What exact inputs produced each important result?

This matters especially for results that we may cite in the rough report. We need to know whether an Opus result was based on a raw source file, a transformed file, or one of our later mappings.

5. Did Opus modify any external data files in place?

If yes, we need to separate original source bytes from derived versions.

6. Are there unpublished scripts or commands needed to reproduce its useful results?

We should capture those now rather than discover later that an early finding cannot be regenerated.

A particularly useful deliverable from Opus would be a single:

opus46/INVENTORY.csv

with columns like:

path

sha256

created_by

artifact_type

status

description

inputs

source_url

source_version

generating_script

command

claims_supported

superseded_by

recommended_location

notes

That would parallel the provenance discipline we already have in gpt56sol/.

One more thing I would ask it explicitly: **identify all conclusions in its earlier reports that it now believes are contradicted or superseded by the later full 56,944-node reconstruction.** That will be very helpful when we write the rough snapshot, because we want historical records preserved without accidentally presenting an early intermediate result as the final conclusion.

Tuesday 11:28 AM

graphsage.utils.py

Python

"Reconstruct every deterministic artifact needed by supervised GraphSAGE and DGL, but do not attempt to regenerate the original stochastic ppi-walks.txt byte-for-byte." I agree with that scope. Separately, I would like to explore that ppi-walks.txt file here to see what we can learn about it.

I also noticed that <https://snap.stanford.edu/graphsage/#datasets> links only to <http://thebiogrid.org/download.php> when describing it's PPI dataset. We learned, that is inaccurate, didn't we?

My reading of ppi-walks is like yours. I saw "If your benchmark/task does not require generalizing to unseen data, we recommend you try setting the "--identity_dim" flag to a value in the range [64,256]." at [williamleif/GraphSAGE](https://github.com/williamleif/GraphSAGE). That is a massive problem given the shared nodes in the training and test sets!

The repo says this about the walks:

- <train_prefix>-walks.txt [optional] --- A text file specifying random walk co-occurrences (one pair per line) (*only for unsupervised version of graphsage)

Does that make it irrelevant for the PPI dataset and its supervised GO predictions? If we need to reproduce this file, the repo says

"To run the model on a new dataset, you need to make data files in the format described above. To run random walks for the unsupervised model and to generate the -walks.txt file) you can use the run_walks function in graphsage.utils." I provided that source file.

Further the manuscript says "Random walks for the unsupervised objective For all settings, we ran 50 random walks of length 5 from each node in order to obtain the pairs needed for the unsupervised loss (Equation 1). Our implementation of the random walks is in pure Python and is based directly on Python code provided by Perozzi et al. [28]." and "All the unsupervised GraphSAGE models and DeepWalk used 20 negative samples with context distribution smoothing over node degrees using a smoothing parameter of 0.75, following [11, 22, 28]. Initial experiments revealed that DeepWalk performed much better with large learning rates, so we swept over rates in the set {0.2, 0.4, 0.8}. For the supervised GraphSAGE methods, we ran 10 epochs for all models. All methods except DeepWalk use batch sizes of 512. We found that DeepWalk achieved faster wall-clock convergence with a smaller batch size of 64." and "Notes on the DeepWalk implementation Existing DeepWalk implementations [28, 11] are simply wrappers around dedicated word2vec code, and they do not easily support embedding new nodes and other variations. Moreover, this makes it

difficult to compare runtimes and other statistics for these approaches. For this reason, we reimplemented DeepWalk in pure TensorFlow, using the vector initializations etc that are described in the TensorFlow word2vec tutorial.⁸ We found that DeepWalk was much slower to converge than the other methods, and since it is 2-5X faster at training, we gave it 5 passes over the random walk data, instead of one. To update the DeepWalk method on new data, we ran 50 random walks of length 5 (as described above) and performed updates on the embeddings for the new nodes while holding the already trained embeddings fixed. We also tested two variants, one where we restricted the sampled random walk “context nodes” to only be from the set of already trained nodes (which alleviates statistical drift) and an approach without this restriction. We always selected the better performing variant. Note that despite DeepWalk’s poor performance on the inductive task, it is far more competitive when tested in the transductive setting, where it can be extensively trained on a single, fixed graph. (That said, Kipf et al [17][18] found that GCN-based approach consistently outperformed DeepWalk, even in the transductive setting on link prediction, a task that theoretically favors DeepWalk.) We did observe DeepWalk’s performance could improve with further training, and in some cases it could become competitive with the unsupervised GraphSAGE approaches (but not the supervised approaches) if we let it run for $>1000\times$ longer than the other approaches (in terms of wall clock time for prediction on the test set); however, we did not deem this to be a meaningful comparison for the inductive task. Note that DeepWalk is also equivalent to the node2vec model [11] with $p = q = 1$."

Ultimately, if we don't need ppi-walks for our supervised PPI task, we can omit it in our reproduction. I do not want to reimplement the walks and like your scoping statement.

Returning to the reproduction, I like this simplified design.

I agree with your MSigDB plan. MSigDB version 6.0 changed the license per https://www.gsea-msigdb.org/gsea/license_terms_list.jsp and https://www.gsea-msigdb.org/gsea/msigdb_license_terms.jsp. We can comment on how the earlier versions give the same results. We could provide a script to transform the private MSigDB file to the minimal version we need as input. We can skip the private CI then.

Please provide a standalone document with timestamp that provides the above repository outline and a short outline of what each subdirectory does. This will guide discussion with my colleague.

Created the standalone Markdown planning document, timestamped **2026-09-01 11:29:03 America/Chicago / 2026-09-01T16:29:03Z**.

Download the reproduction repository outline

SHA-256:

6dfdd2b2e21e22fe757e212ccb66f7ab8a2f809d6a862e5aa694330b700ae340

The document includes the agreed narrow scope, complete proposed directory tree, a short description of every directory and module, the Pixi/Snakemake/pytest design, the minimal MSigDB preparation approach, the boundary between reconstruction and validation, ignored output directories, and the initial implementation order.

It also records two important scope conclusions:

- ppi-walks.txt is explicitly documented as optional and only for unsupervised GraphSAGE. The supervised trainer loads the dataset without walks, whereas the unsupervised trainer requests them. The supplied utility source likewise defaults load_walks=False, reads the file only when requested, and generates pairs using stochastic random.choice.
- The GraphSAGE project page's PPI "Source" link points only to BioGRID. The document describes this as **incomplete and potentially misleading provenance**, rather than wholly false: BioGRID contributed interactions, but the released tissue graphs trace directly to OhmNet and its broader composite interactome.

The identity-feature recommendation is noted as a separate leakage concern for the later ML study, not as part of the deterministic data-reconstruction package.

REPRODUCTION_REPOSITORY_OUTLINE_20260901T162903Z.mdDocumentOpen file

 Citations





[GitHub](#)





[SNAP](#)

[GraphSAGE](#)

 graphsage.utils.py

graphsage.utils.py

Update that design design to focus only on the reproduction. My BioGRID questions and ppi walk questions were secondary.

I revised the design so it now focuses exclusively on the **core deterministic reproduction**.

The updated version removes the extended discussions of:

- BioGRID and upstream PPI provenance audits;
- ppi-walks.txt implementation details;
- alternative annotation-source investigations;
- historical GOA and MSigDB release screens;
- column-order model searches;
- leakage experiments;
- literature discrepancies and dataset-impact analysis.

It retains only a brief scope boundary noting that auxiliary stochastic artifacts outside supervised GraphSAGE and DGL are not reconstruction targets.

Core Reproduction Design — Revision Summary

The revised repository design is limited to one objective:

Starting from independently acquired external sources and explicitly documented recovered decisions, reconstruct the deterministic GraphSAGE PPI artifacts used by supervised GraphSAGE, transform them into the DGL representation, and verify them against independently downloaded reference outputs.

The core workflow includes:

1. External-source acquisition and checksum validation.
2. OhmNet-to-GraphSAGE node and topology reconstruction.
3. MSigDB-to-GraphSAGE feature reconstruction.
4. GOA-to-GraphSAGE label reconstruction.

5. Deterministic GraphSAGE artifact assembly.
6. DGL artifact generation.
7. Exact or explicitly tolerated comparisons with released targets.
8. Run-level provenance and validation summaries.

Exploratory audits, literature analysis, leakage experiments, impact assessment, and stochastic unsupervised artifacts remain outside reproduction/.

Download the updated core reproduction outline

SHA-256:

43549c103272b06e6ff10e7228be0687fe1db24793ce91f21650cfed1c62b361

This version supersedes

REPRODUCTION_REPOSITORY_OUTLINE_20260901T162903Z.md.

REPRODUCTION_REPOSITORY_OUTLINE_CORE_20260901T164642Z.mdDocumentOpen
file

Yesterday 8:54 AM

GraphSAGE-a0fdef9.zip

Zip Archive

The reproduction strategy looks good now. Before we start implementing, I am curious about a few other things. What could we do to attempt to reproduce the PPI walks? I understand the non-determinism. However, we have the original source code, and idea of what version of Python and packages may have been used, and critically the outputs. Try many strategies to see whether you can reproduce the walks provided in the GraphSAGE data. Come up with a scheme to quickly abort if unsuccessful so you can try more ideas and iterations. Those could be arbitrary seeds, setting common random seeds, and other ideas. Be exhaustive. There are technically a finite number of ways this could have been run. Think carefully about how to best prune and explore that search space. Make sure you have access to the full GraphSAGE code. It is attached here.

We are at the maximum conversation length. You need to produce a fully detailed artifact with our full conversation and all critical data files that you produced. I can resupply input data files as needed so omit those. I need a seed for a new conversation that will functionally resume from where we are now so I can work on the reproduction plan with full context.

I created a complete handoff bundle designed specifically to resume this project in a new conversation.

Download the full new-conversation handoff bundle

SHA-256:

ad8e0a907544948ac4970ba6c9c9182f1ff8ae7270d7e395a72aaef154a64b55

Download the checksum file

The bundle contains a detailed functional reconstruction of this conversation, the current scientific state, all major corrections and unresolved questions, the agreed clean reproduction architecture, the pending ppi-walks.txt side investigation, and the most important current data/result artifacts. It intentionally omits the large external input datasets because you said you can resupply those.

The best files to use in the new conversation are:

- Full project handoff — detailed project/conversation reconstruction.
- Ready-to-paste new conversation seed — concise enough to use as the initial prompt.
- Latest core reproduction design
- Critical artifact index with SHA-256 hashes
- Complete internal checksums

I also included the earlier 57 MB comprehensive GPT-5.6 artifact archive inside the handoff, so obscure intermediate analyses remain recoverable without re-uploading every old batch separately.

One limitation: I cannot export an exact byte-for-byte transcript of the ChatGPT UI conversation. The handoff therefore contains a **detailed functional reconstruction** of the full discussion plus the chronological investigation report and the retained scientific artifacts. It is intended to preserve everything necessary to continue the work faithfully.

For the next conversation, I recommend uploading the handoff ZIP and starting with the text from NEW_CONVERSATION_SEED_20260904T115559Z.md. The seed explicitly records that the main next phase is the clean reproduction implementation, while the exhaustive ppi-walks.txt reconstruction attempt remains an optional side investigation.

You've reached the maximum length for this conversation, but you can keep talking by starting a new chat.

[Start new chat](#)

[GraphSAGE_PPI_new_conversation_handoff_20260904T115559Z.zip.sha256.txt](#)Document
Open file

[FULL_PROJECT_HANDOFF_20260904T115559Z.md](#)DocumentOpen file

[NEW_CONVERSATION_SEED_20260904T115559Z.md](#)DocumentOpen file

[REPRODUCTION_REPOSITORY_OUTLINE_CORE_20260901T164642Z.md](#)DocumentOpen
file

[CRITICAL_ARTIFACT_INDEX_20260904T115559Z.csv](#)SpreadsheetOpen file

[SHA256SUMS_20260904T115559Z.txt](#)DocumentOpen file

[Show less](#)